TEACHING WALLAH

Teaching Exam

Subject – Computer Science

Data Structure and Algorithm

Lecture No -6

By-Satyendra Chaudhary Sir

Topics to be Covered

Topic

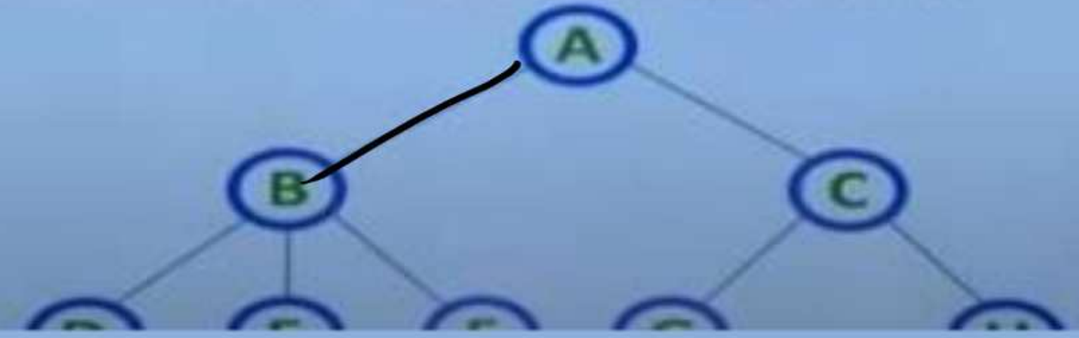
Trees & Graphs

- coding
- Traversing
- Searching
- Matrix

Data Structure	Access	Insert	Delete	Use Case
Array	O(1)	O(n)	O(n)	Random access, fixed size
Linked List	O(n)	O(1)/O(n)	O(1)/O(n)	Dynamic size, insertion-heavy
Stack	O(1)	O(1)	O(1)	Recursion, parsing
Queue	O(1)	O(1)	O(1)	Task scheduling
Hash Table	O(1)*	O(1)*	O(1)*	Fast lookup by key
Tree (BST)	O(log n)	O(log n)	O(log n)	Sorted data, hierarchy
Heap	O(n)	O(log n)	O(log n)	Priority handling
Graph	Varies	Varies	Varies	Network modeling
Trie	O(L)	O(L)	O(L)	Prefix search

Nodes (N)
Edges (N-1) Trees.

- **Root Node**: The top-most node, serving as the origin of the tree.
- **Edge**: The connection between two nodes. A tree with 'N' nodes has exactly 'N-1' edges.
- **Parent Node**: A node that has one or more children.
- **Child Node**: A node that descends from a parent node.
- **Leaf (External) Node**: A node with no children.
- **Internal Node**: A node with at least one child.
- **Degree of Node**: The number of children a node has.
- **Level/Depth**: Indicates the step count from the root (Level 0) to a particular node.
- **Path**: A sequence of nodes connected by edges.
- **Subtree**: A tree formed from a child node and its descendants.

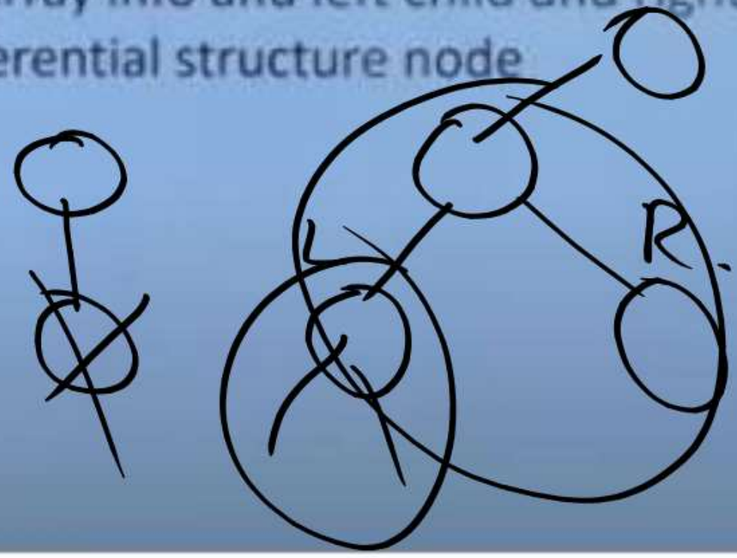


degree -

Binary tree

- A **binary tree** is a type of tree data structure where each node can have at most **two children**, commonly referred to as **left** and **right** child nodes. It is structured as a set of nodes with the following properties:
- **Empty Tree:** A binary tree can be empty (no nodes).
- **Root Node:** If not empty, the tree has a unique root node (R).
- **Subtrees:** The remaining nodes are partitioned into two disjoint subtrees, **T1** (left) and **T2** (right).
- Representation of tree in memory
  - Sequential representation – using an array info and left child and right child
  - Linked Representation – using self-referential structure node

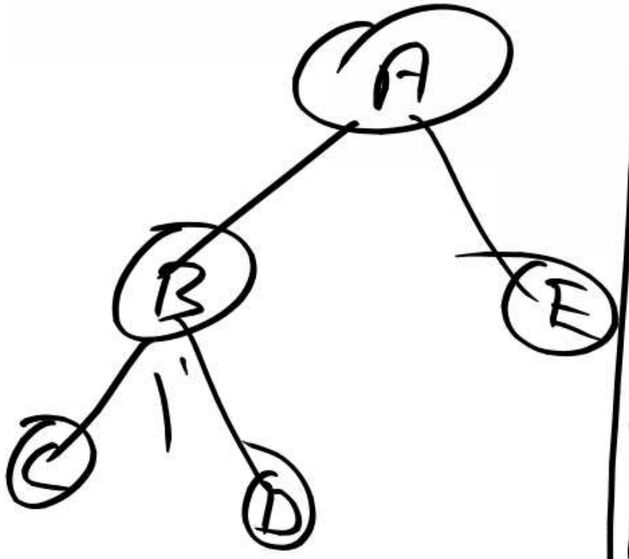
```
struct node {
   int data;
   struct node* left;
   struct node* right;
}
```

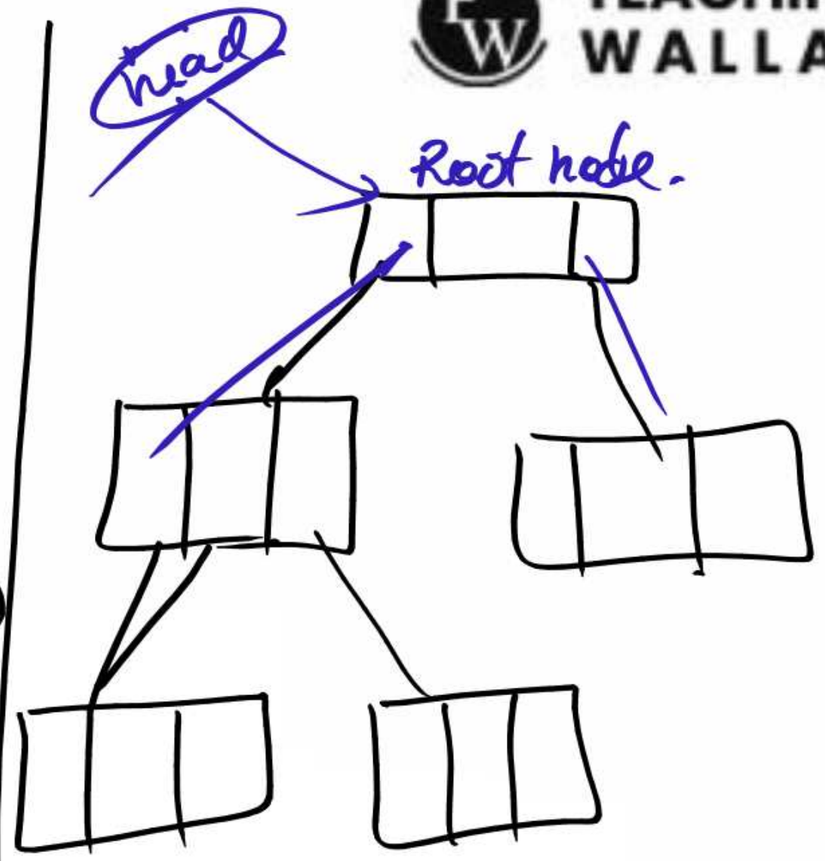


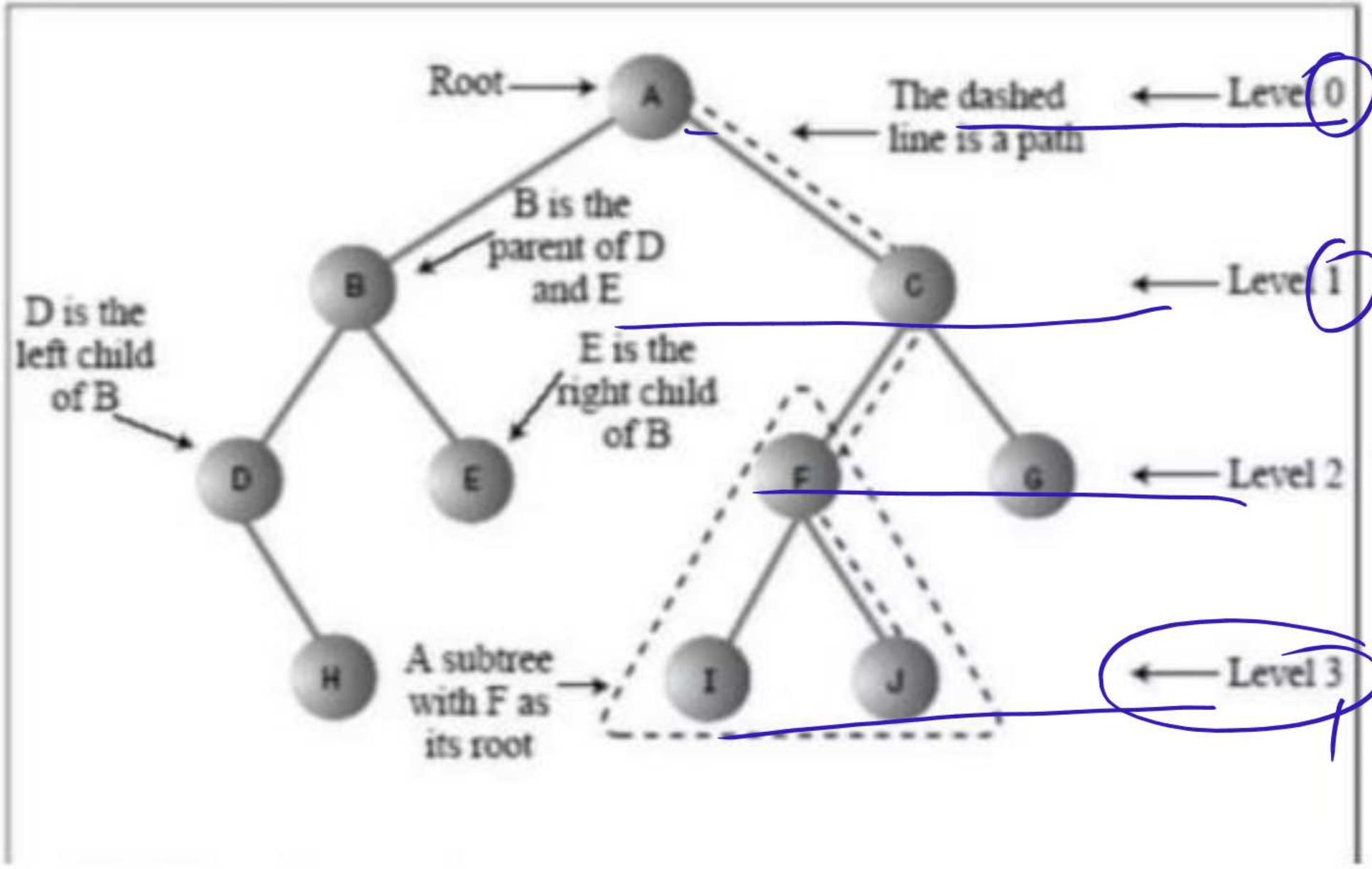
Implementation

Arrays

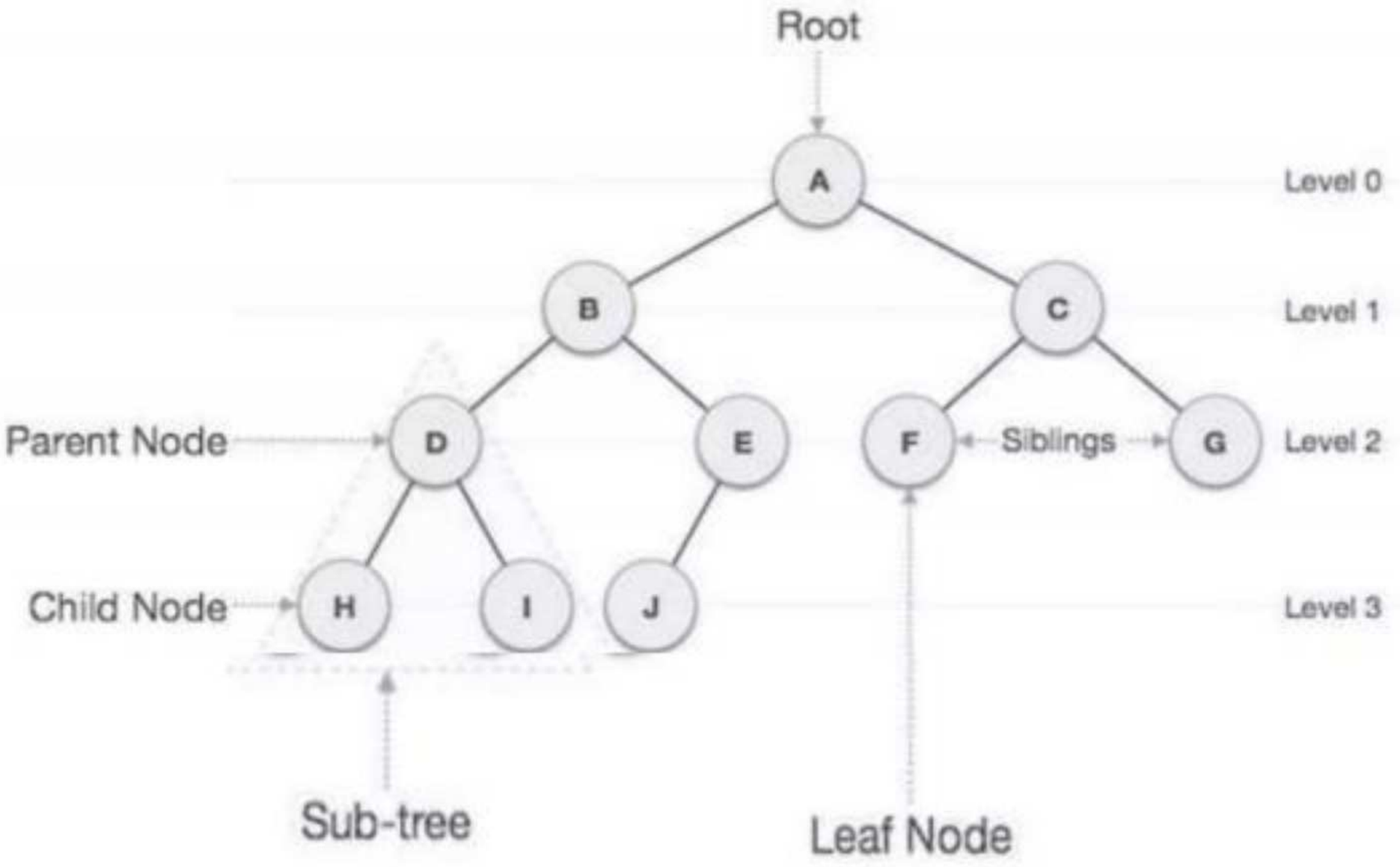
	Node	Left child	Right child
0	A	B	E
1	B	C	D
2	E		
3	C	X	X
4	D		

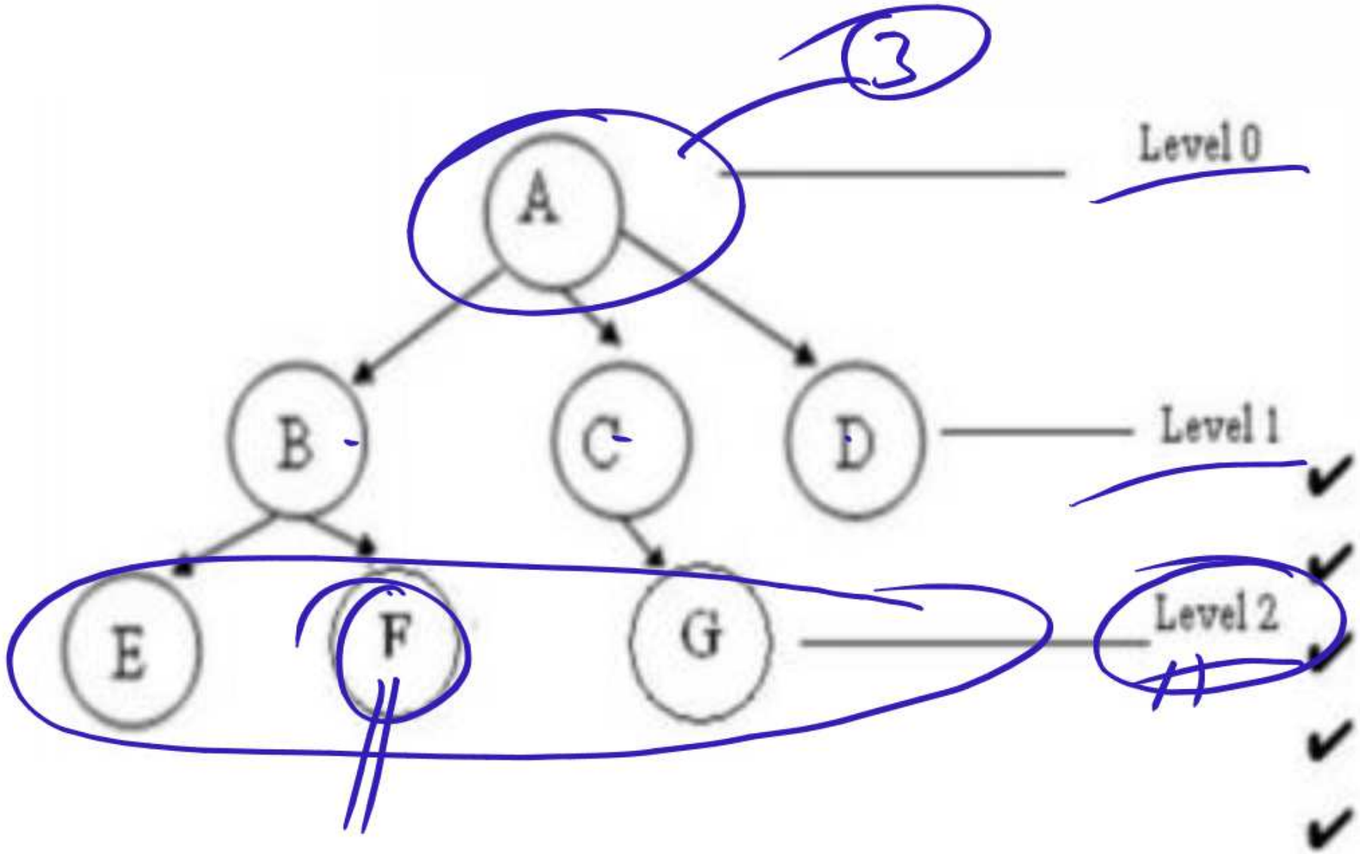






Height = 3





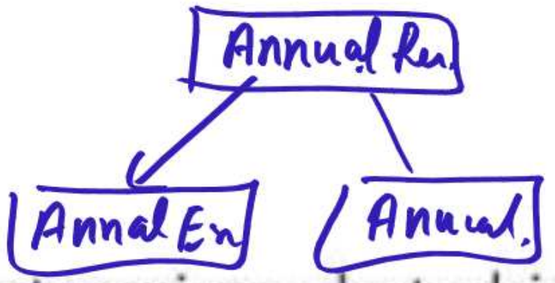
- A is the root node
- B is the parent of E and F
- D is the sibling of B and C
- E and F are children of B
- E, F, G, D are external nodes or leaves
- A, B, C are internal nodes
- Depth of F is 2
- the height of tree is 2
- the degree of node A is 3
- The degree of tree is 3

Characteristics of trees

- Non-linear data structure
- Combines advantages of an ordered array
- Searching as fast as in ordered array
- Insertion and deletion as fast as in linked list
- Simple and fast

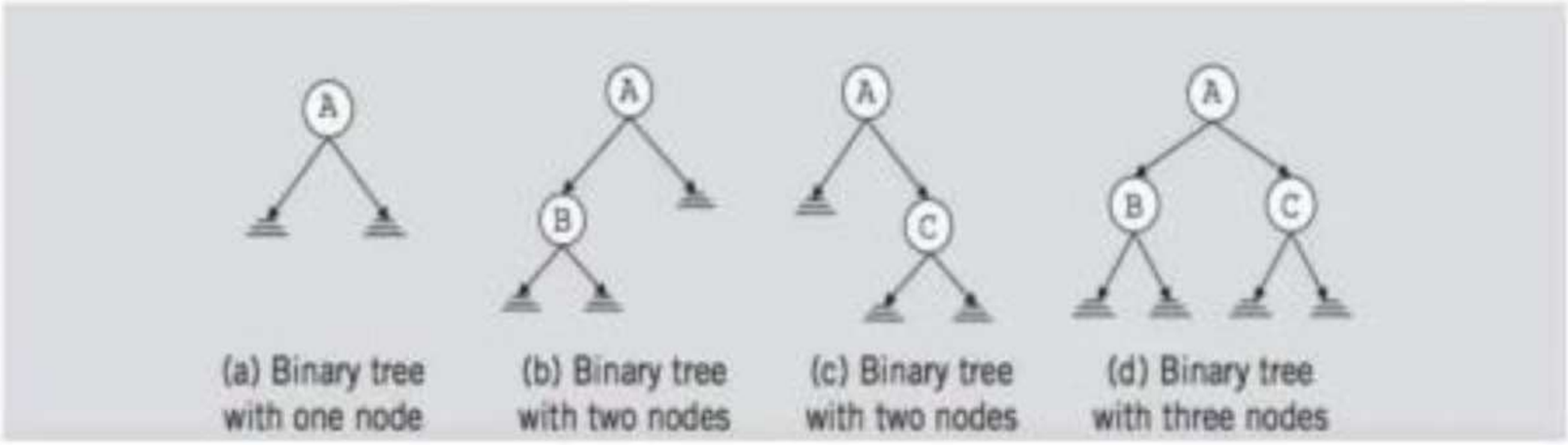
Application

- Directory structure of a file store
- Structure of an arithmetic expressions
- Used in almost every 3D video game to determine what objects need to be rendered.
- Used in almost every high-bandwidth router for storing router-tables.
- used in compression algorithms, such as those used by the .jpeg and .mp3 file-formats.



Binary Trees

- A **binary tree**, T, is either empty or such that
  - *I.* T has a special node called the **root** node
  - *II.* T has two sets of nodes L_T and R_T, called the **left subtree** and **right subtree** of T, respectively.
  - *III.* L_T and R_T are binary trees.

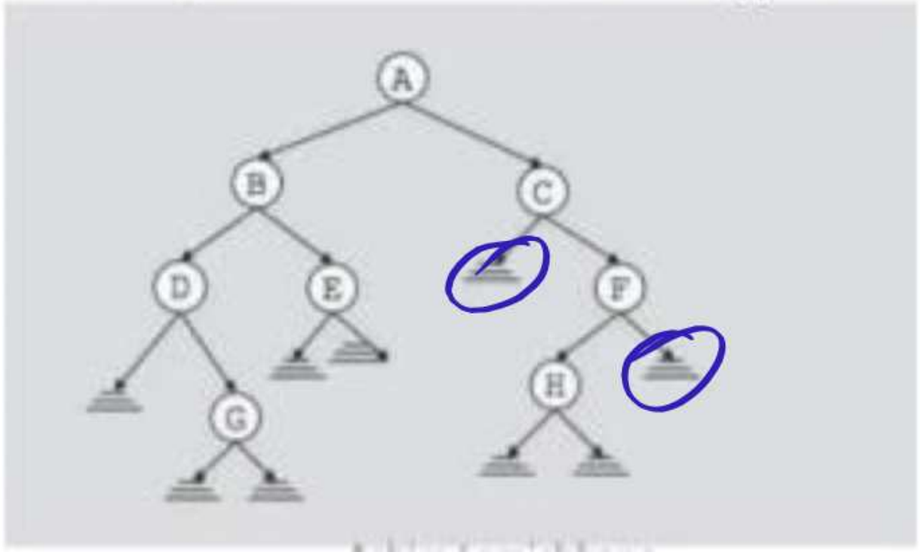


Binary Tree

- The root node of this binary tree is A.

- The left sub tree of the root node, which we denoted by L_A, is the set L_A = {B,D,E,G} and the right sub tree of the root node, R_A is the set R_A={C,F,H}

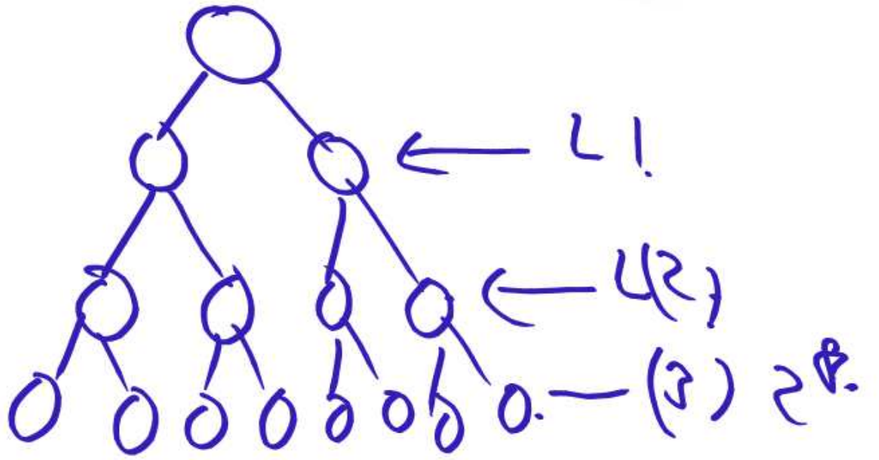
- The root node of L_A is node B, the root node of R_A is C and so on



Binary Tree Properties

- If a binary tree contains m nodes at level L, it contains at most 2m nodes at level L+1

- Since a binary tree can contain at most 1 node at level 0 (the root), it contains at most 2^L nodes at level L.

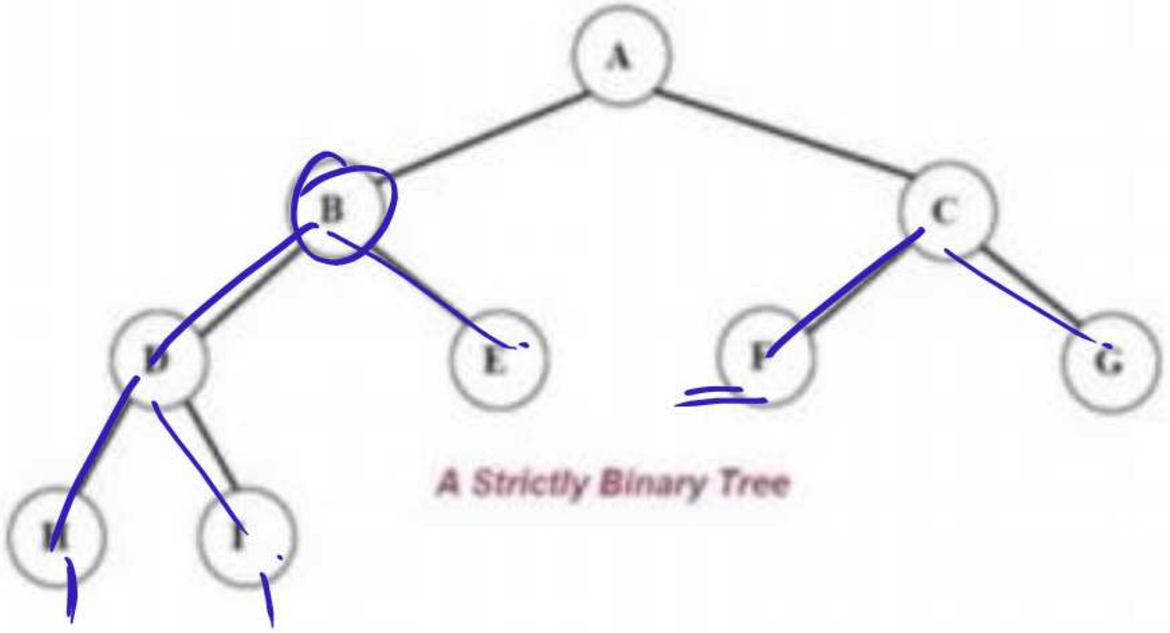


Types of Binary Tree

- Complete binary tree
- Strictly binary tree
- Almost complete binary tree

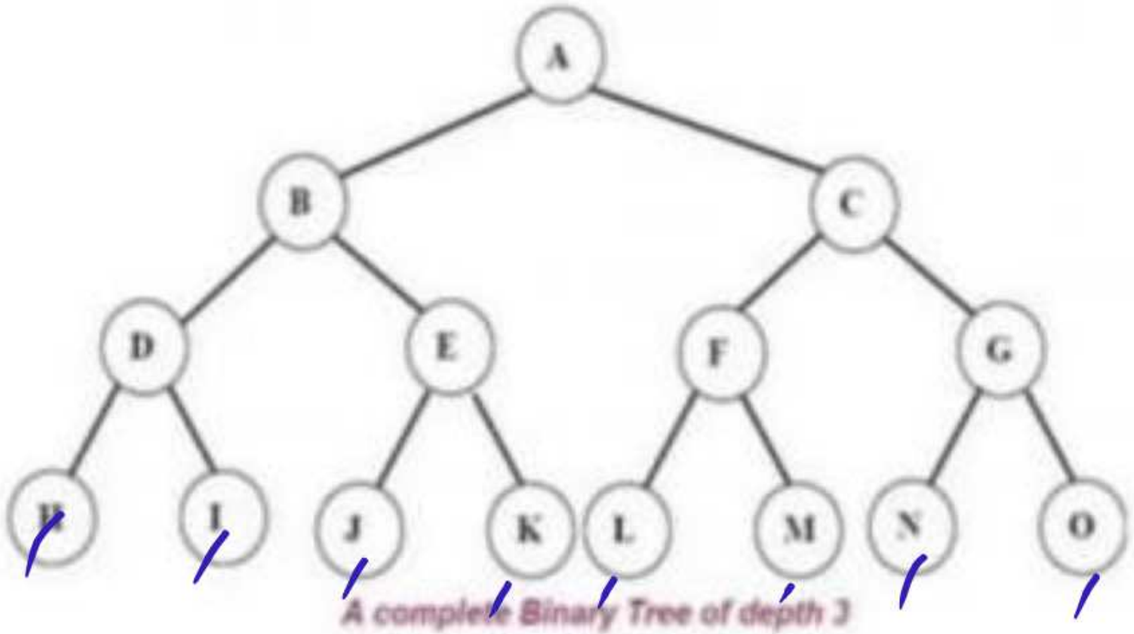
- **Or, to put it another way,** all of the nodes in a strictly binary tree are of degree zero or two, never degree one.

- A strictly binary tree with N leaves always contains 2N – 1 nodes.



A Strictly Binary Tree

- A ***complete binary tree*** of depth **d** is called strictly binary tree if all of whose leaves are at level **d**.
- A complete binary tree has 2^d nodes at every depth **d** and 2^d -1 non leaf nodes



A complete Binary Tree of depth 3

- An almost complete binary tree is a tree where for a right child, there is always a left child but for a left child there may not be a right child.

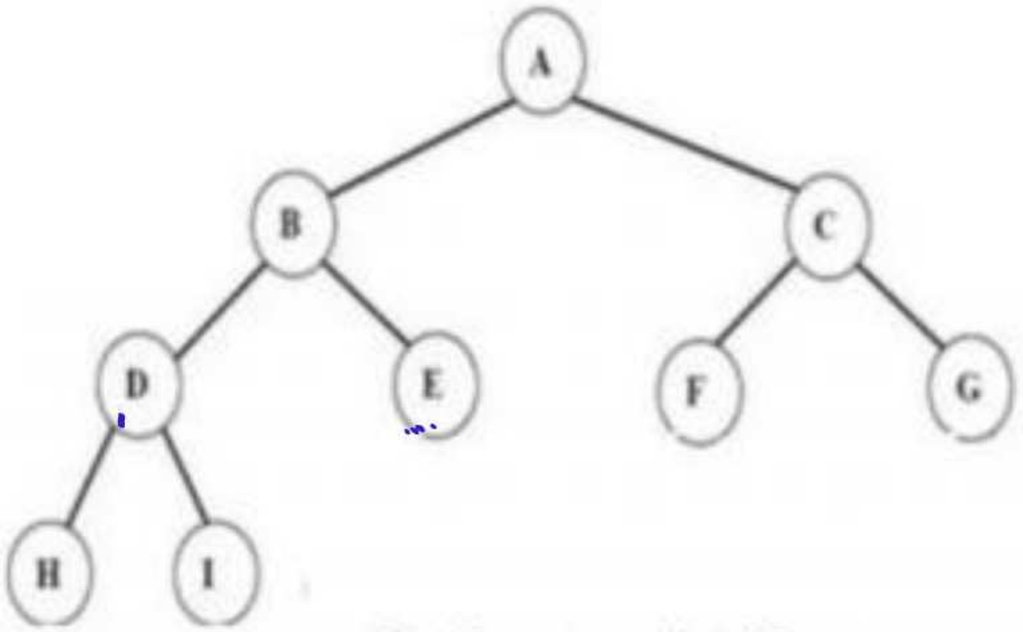


Fig Almost complete binary tree.

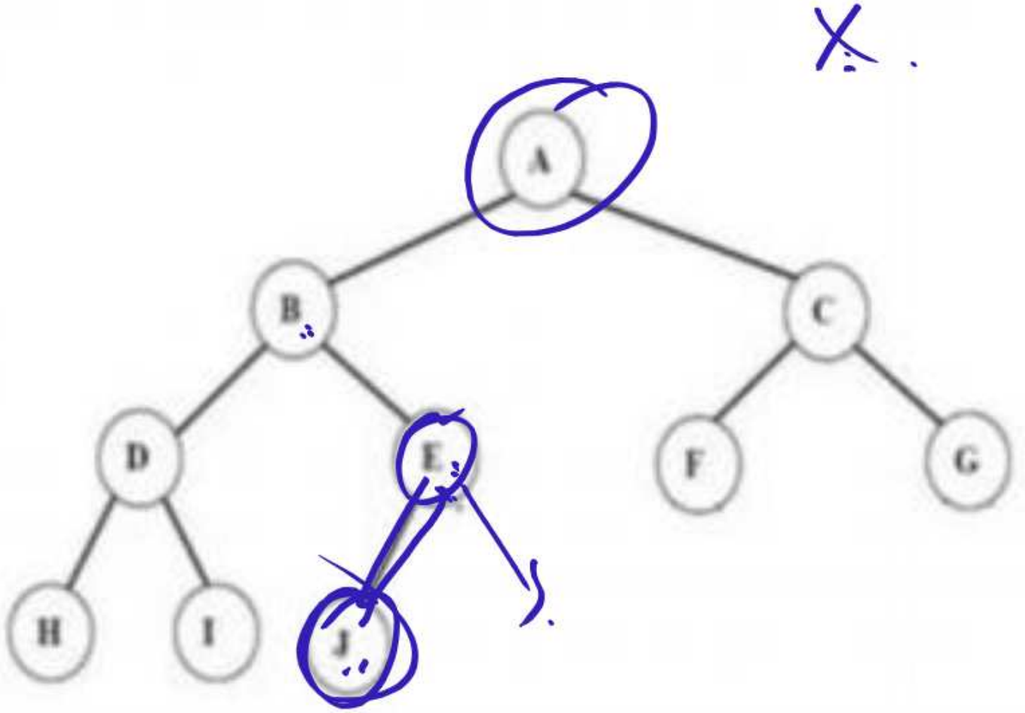
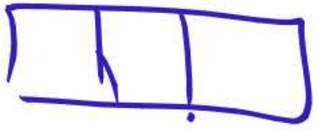


Fig Almost complete binary tree but not strictly
Since node E has a left son but not a right son.

Operations on Binary tree:

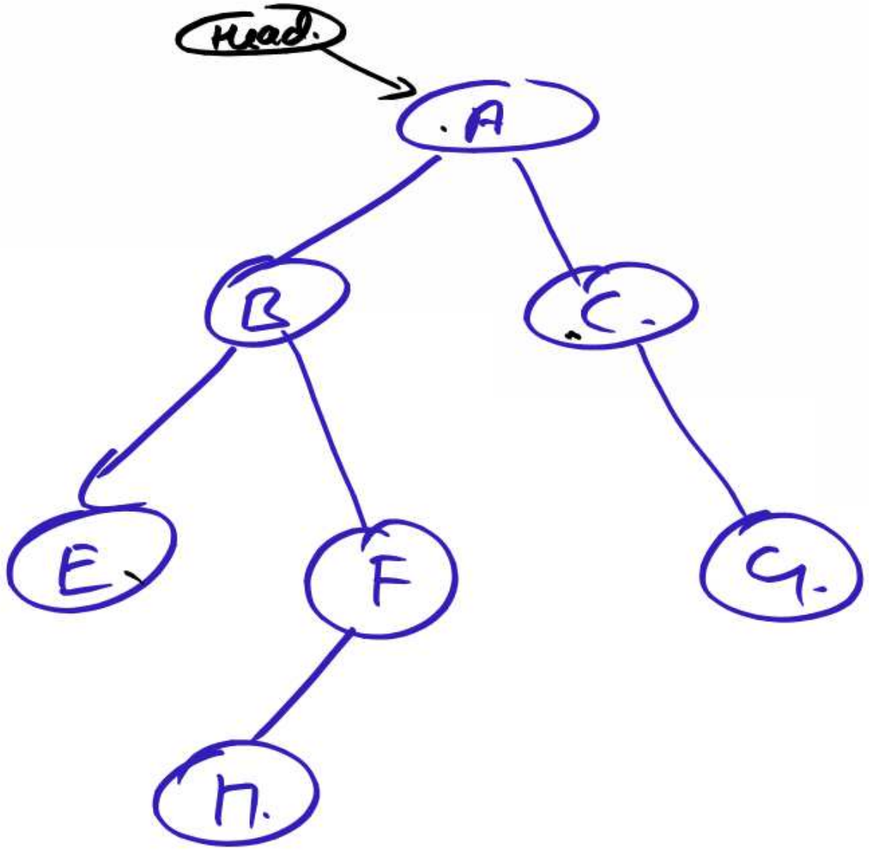
- **father(n,T):** Return the parent node of the node n in tree T. If n is the root, NULL is returned.
- **LeftChild(n,T):** Return the left child of node n in tree T. Return NULL if n does not have a left child.
- **RightChild(n,T):** Return the right child of node n in tree T. Return NULL if n does not have a right child.
- **Info(n,T):** Return information stored in node n of tree T (ie. Content of a node).
- **Sibling(n,T):** return the sibling node of node **n** in tree T. Return NULL if n has no sibling.



Preorder

Postorder

Inorder



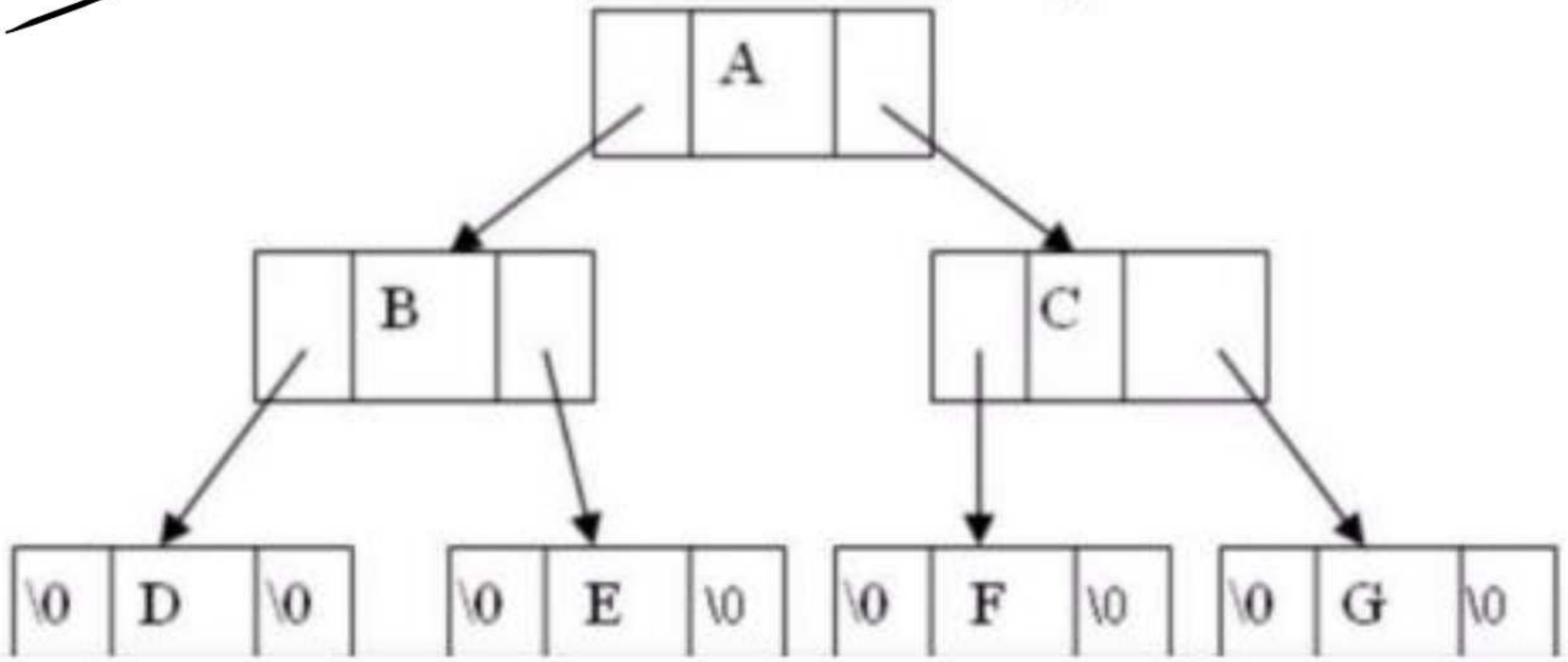
(3)

A B E F H C G

- ✔ **Root(T):** Return root node of a tree if and only if the tree is nonempty.
- ✔ **Size(T):** Return the number of nodes in tree T
- ✔ **MakeEmpty(T):** Create an empty tree T
- ✔ **SetLeft(S,T):** Attach the tree S as the left sub-tree of tree T
- ✔ **SetRight(S,T):** Attach the tree S as the right sub-tree of tree T.
- ✔ **Preorder(T):** Traverses all the nodes of tree T in preorder.
- ✔ **postorder(T):** Traverses all the nodes of tree T in postorder
- ✔ **Inorder(T):** Traverses all the nodes of tree T in inorder.

C representation for Binary tree:

```
struct bnode
{
        int info;
        struct bnode  *left;
        struct bnode  *right;
};
struct bnode *root=NUI
```



Tree traversal

- Traversal is a process to visit all the nodes of a tree and may print their values too.

- All nodes are connected via edges (links) we always start from the root (head) node.

- There are three ways which we use to traverse a tree
  - In-order Traversal
  - Pre-order Traversal
  - Post-order Traversal

- Pre-order

<root><left><right>

Root L. R.

- In-order

<left><root><right>

L Root R.

- Post-order

<left><right><root>

L R Root

- The preorder traversal of a nonempty binary tree is defined as follows:
  - Visit the root node
  - Traverse the left sub-tree in preorder
  - Traverse the right sub-tree in preorder

A B D H I E F C G

Root Node is 1st node

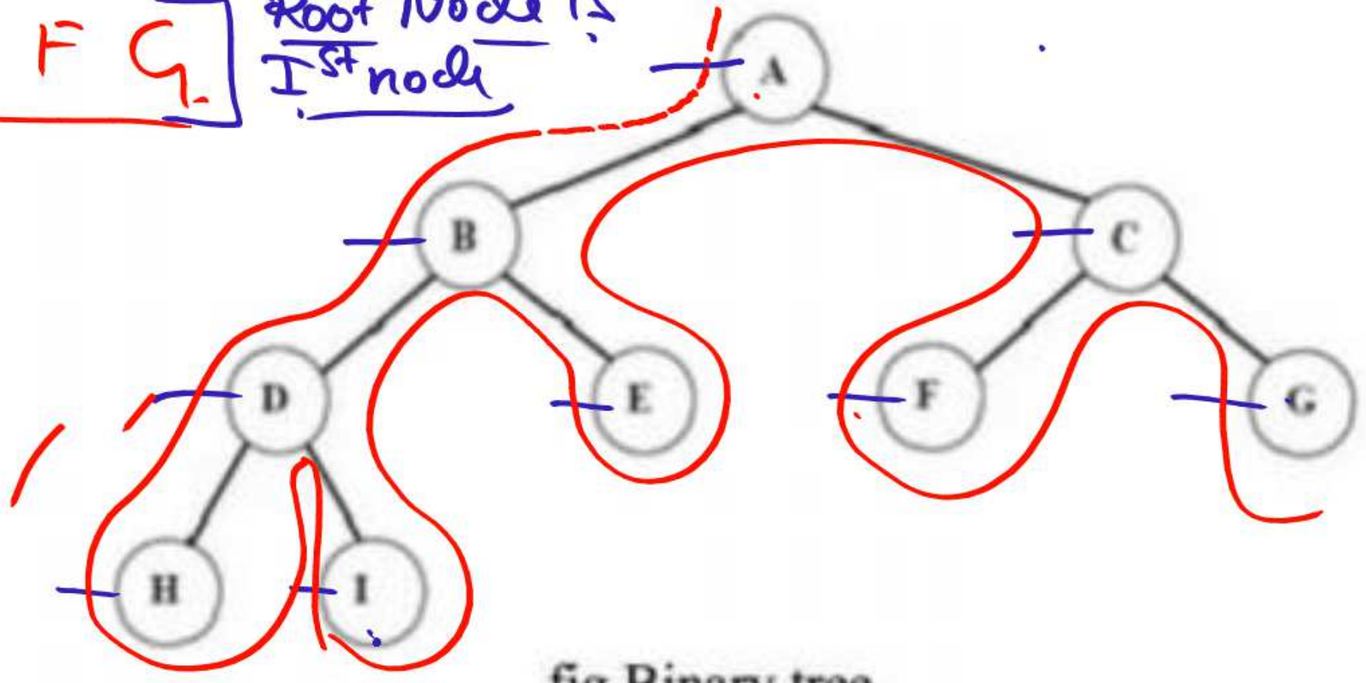
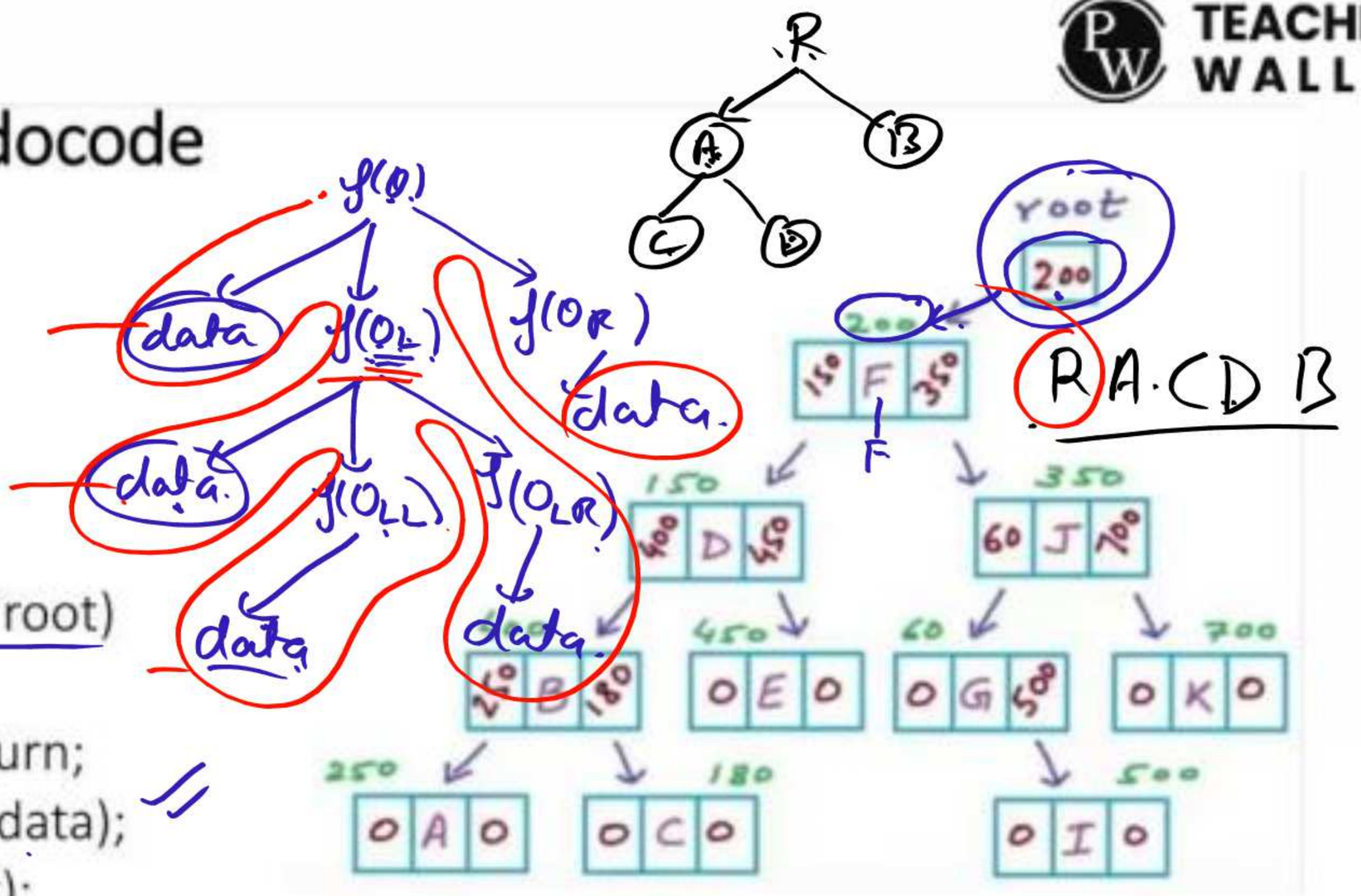


fig Binary tree

The preorder traversal output of the given tree is: A B D H I E C F G

Pre-order Pseudocode

```
struct Node{
    char data;
    Node *left;
    Node *right;
}
void Preorder(Node *root)
{
    if (root==NULL) return;
    printf ("%c", root->data);
    Preorder(root->left);
     Preorder(root->right);
 }
```



- The in-order traversal of a nonempty binary tree is defined as follows:
  - Traverse the left sub-tree in in-order
  - Visit the root node
  - Traverse the right sub-tree in inorder

Post order.

H, I, D, E, B, F, G, C, A

Root Node appears in last

- The in-order traversal outpu of the given tree is
  H D I B E A F C G

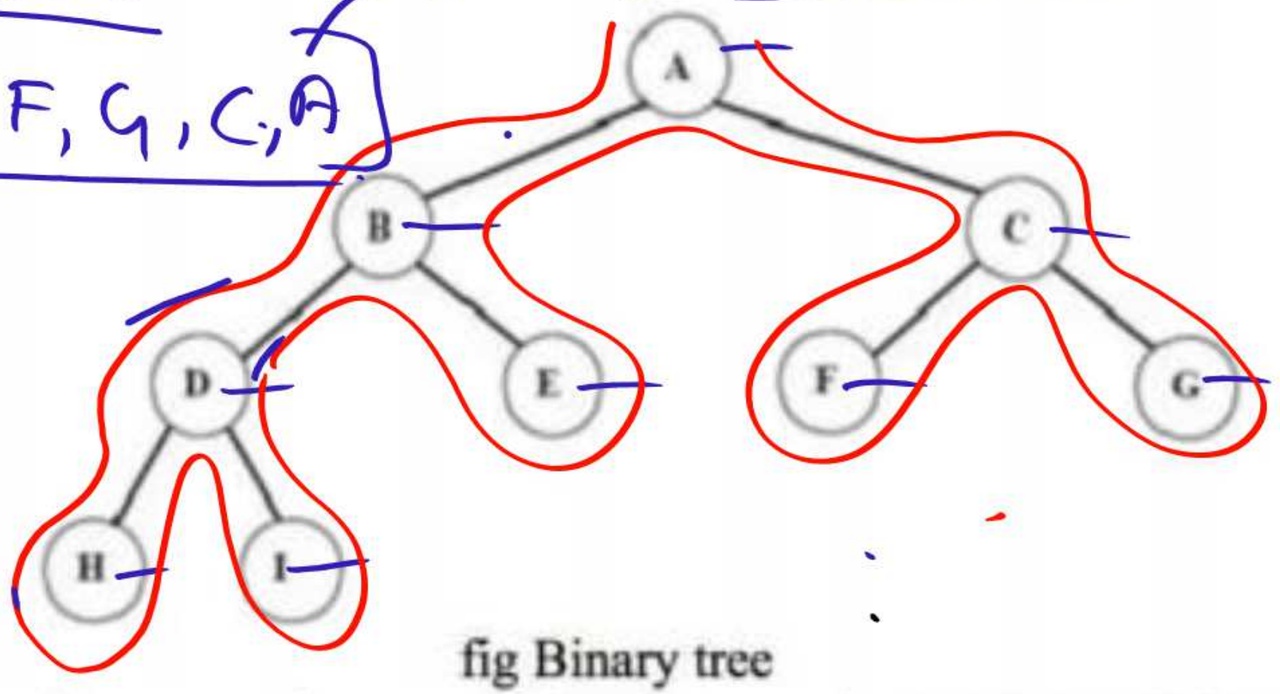


fig Binary tree

Post

1) MBCAFHPYK

Pre 2) KAMCBYPFH

3) MABCKYFPH

In

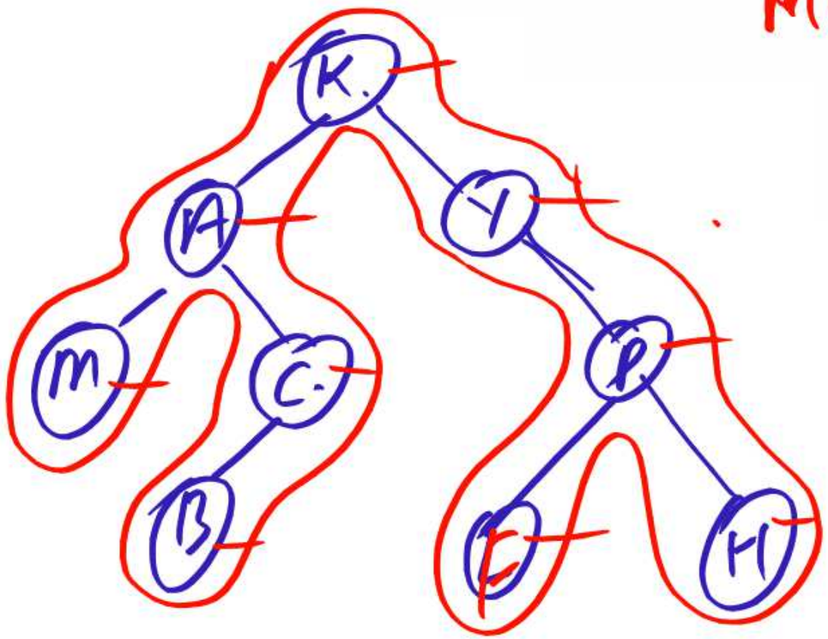
KAMCBYPFH

Inorder, Preorder, Postorder.

Tree → In, Pre → Post

→ In, Post.

MBCAFHPYK.

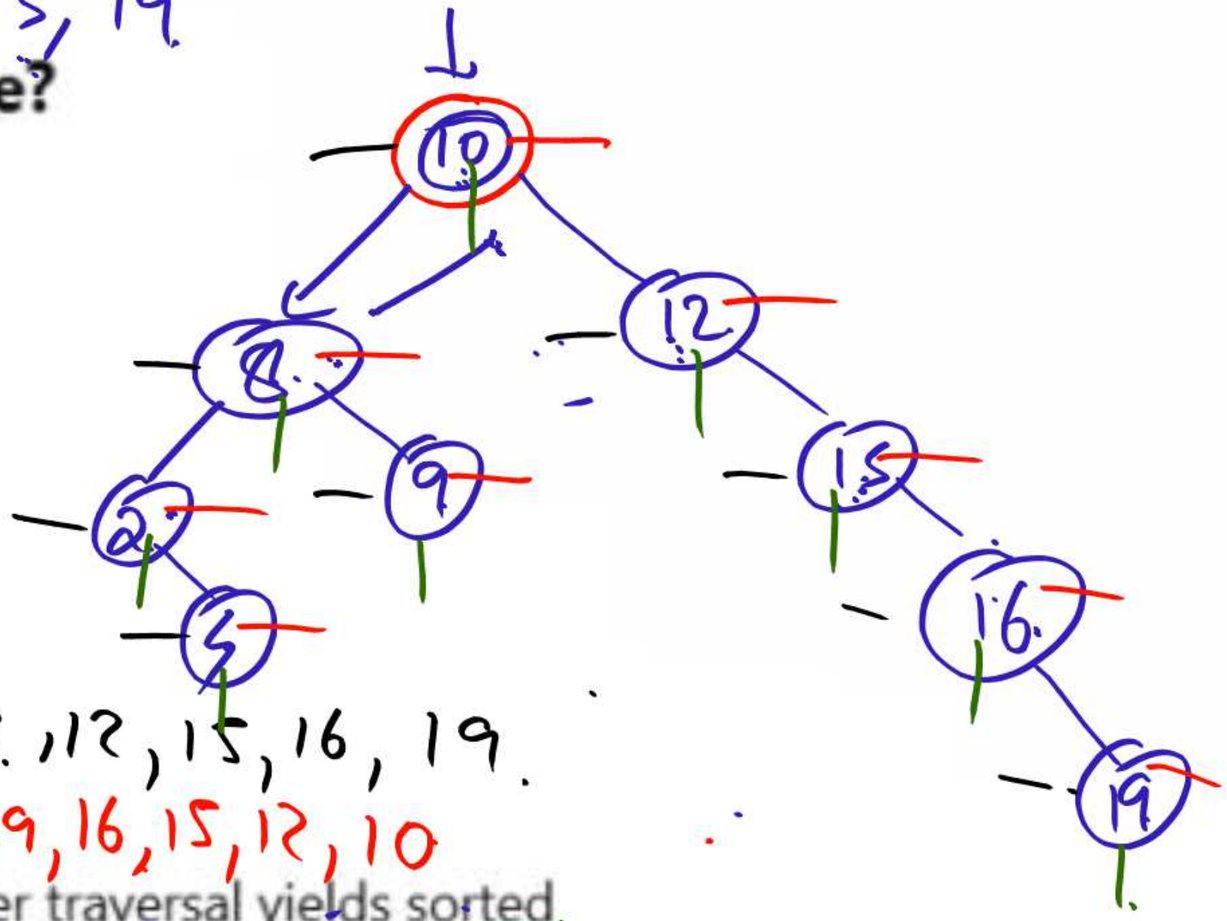


TEACHING WALLAH

10 8, 9, 12, 15, 16, 2, 5, 19.

: What is a Binary Search Tree?

- A **binary tree** with these properties:
  - Left subtree: keys < node key
  - Right subtree: keys > node key
- No duplicate keys



Pre order.

10, 8, 2, 5, 9, 12, 15, 16, 19.

BST Properties

Post order: 5, 2, 9, 8, 19, 16, 15, 12, 10

✅ Binary tree with ordering constraints ✅ In-order traversal yields sorted sequence ✅ Height affects performance

Inorder: 2, 5, 8, 9, 10, 12, 15, 16, 19

Pre order of BST

12, 8, 6, 2, 7, 9, 10, 16, 15, 19, 17, 20.

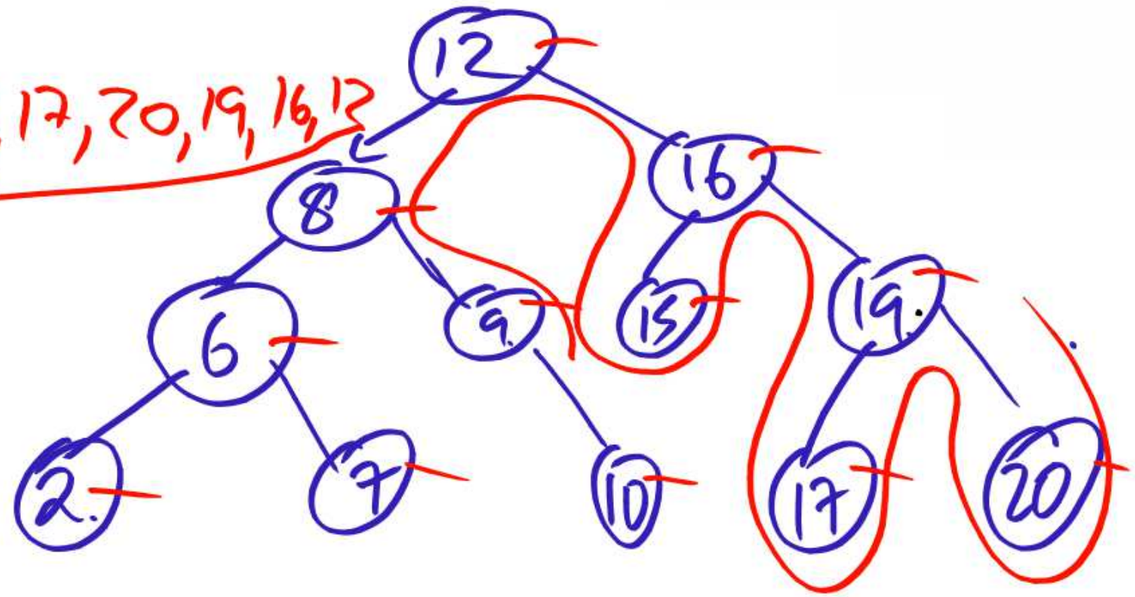
Inorder → 2, 6, 7, 8, 9, 10, 12, 15, 16, 17, 19, 20.

Post order = ?

2, 7, 6, 10, 9, 8, 15, 17, 20, 19, 16, 12

Inorder

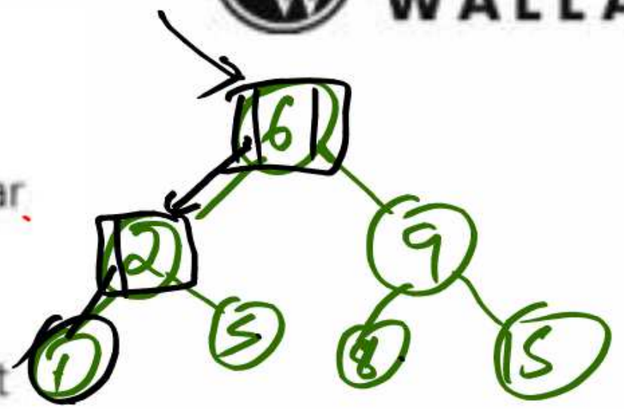
Preorder → Post order



Advantages of BST

✅ Dynamic insertion and deletion ✅ Sorted data retrieval ✅ Faster than linear structures in average case

Disadvantages: ❌ Unbalanced trees degrade to O(n) ❌ No guaranteed height balance without AVL/Red-Black Trees



O(log n)

15, 9, 8, 6, 5, 2, 1,

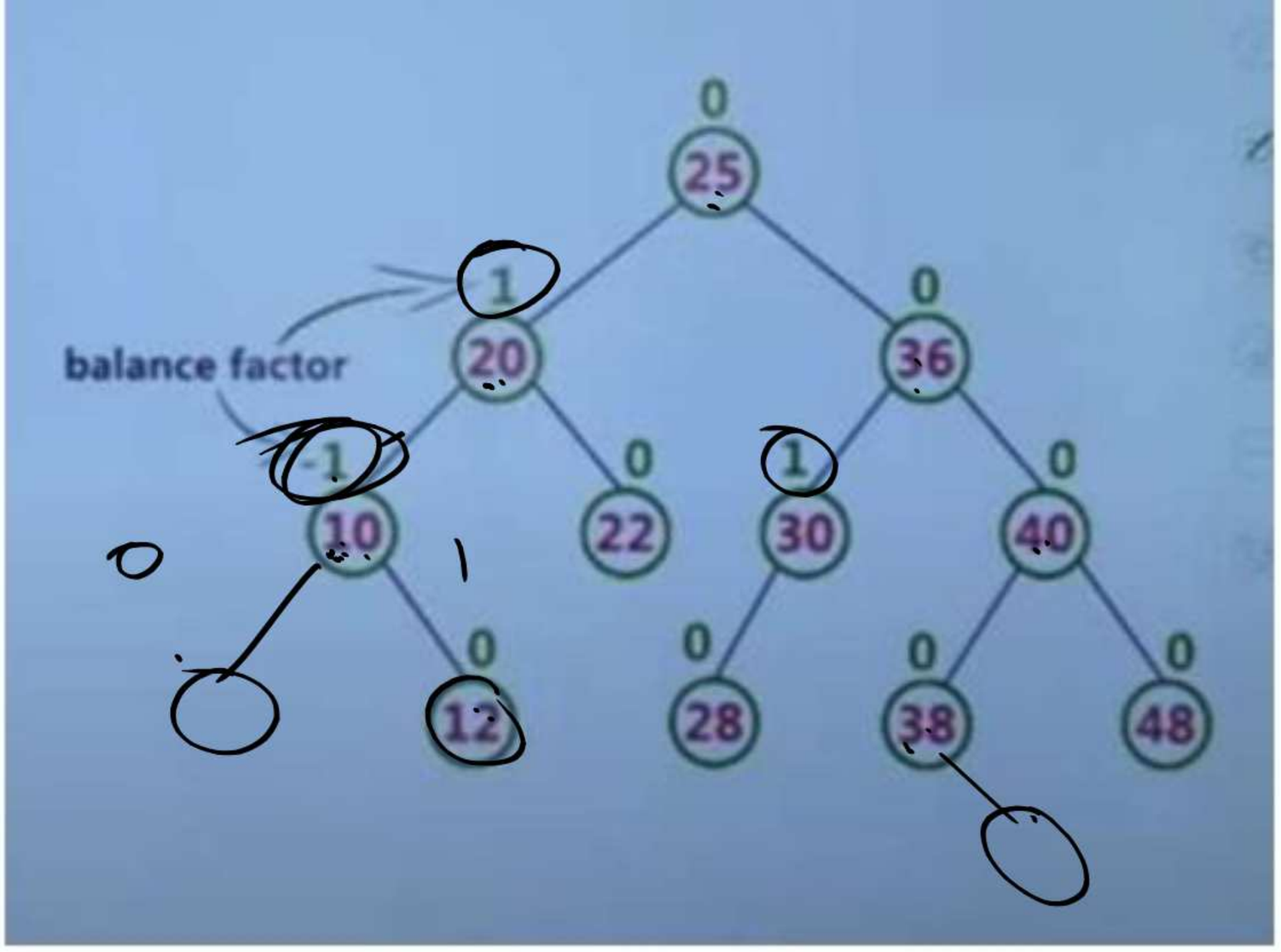
6, 2, 5, 1, 9, 8, 15

: Conclusion

- BSTs enable efficient dynamic ordered data storage
- Important to understand balancing to maintain performance

✅ Master search, insert, and delete operations

Algorithm	Average	Worst case
Space	O(n)	O(n)
Search	O(log n)	O(n)
Insert	O(log n)	O(n)
Delete	O(log n)	O(n)



AVL

0, -1, +1

Left subtree - Right

What is an AVL Tree?

- A **self-balancing binary search tree**
- For every node:
  - Balance Factor = Height(left subtree) - Height(right subtree)
  - Must be -1, 0, or +1

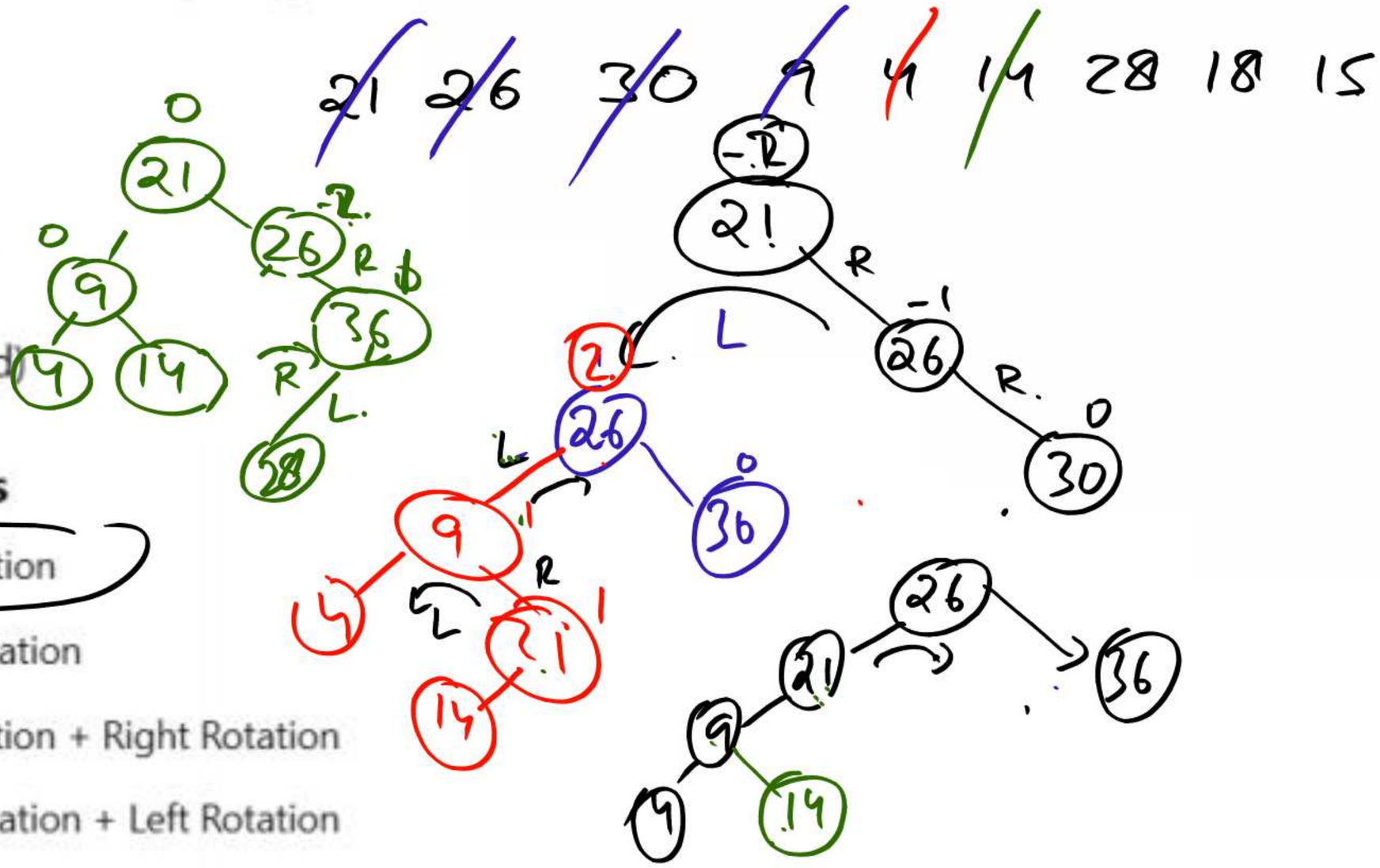
Why AVL Trees?

- Regular BSTs can become unbalanced
- Worst-case BST time complexity: O(n)
- AVL Trees guarantee O(log n) time for:
  - Search
  - Insertion
  - Deletion

- **Balance Factor = Height(left) - Height(right)**

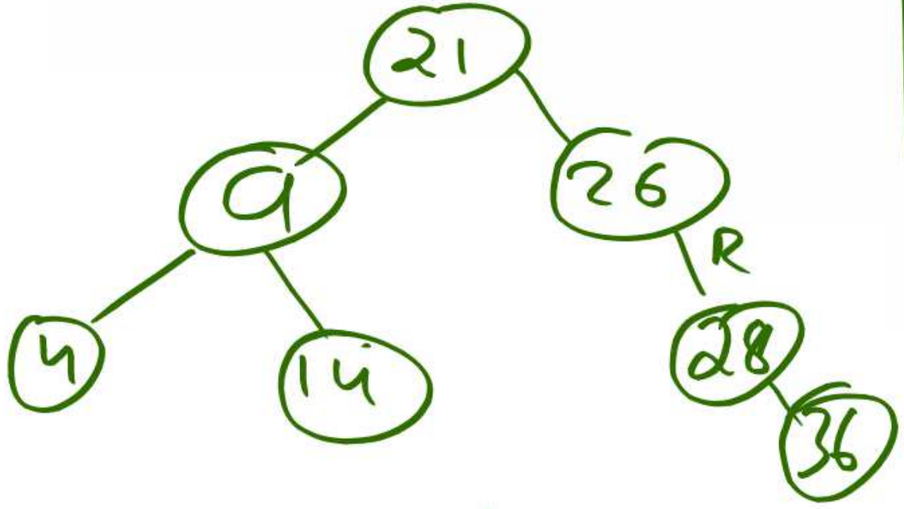
Examples:

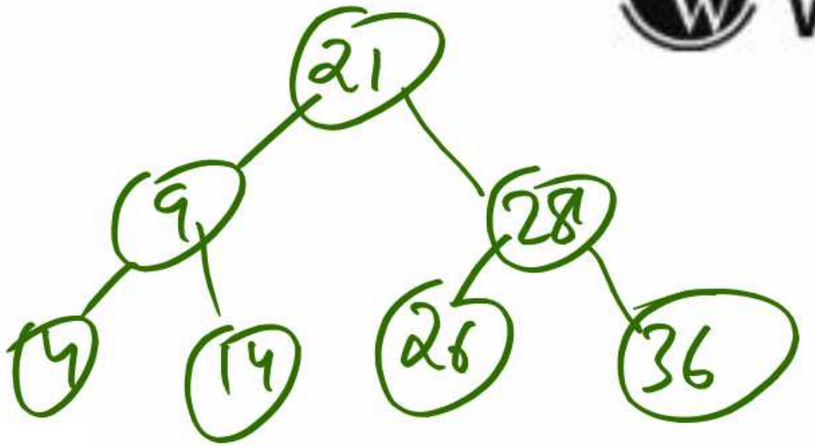
- BF = 0 (balanced)
- BF = +1 or -1 (balanced)
- BF = +2 or -2 (unbalanced)



Insertion Cases

- **Left-Left Case**: Right Rotation
- **Right-Right Case**: Left Rotation
- **Left-Right Case**: Left Rotation + Right Rotation
- **Right-Left Case**: Right Rotation + Left Rotation





1. How many children does a binary tree have?

a) 2

b) any number of children

c) 0 or 1 or 2

d) 0 or 1

2. What is/are the disadvantages of implementing tree using normal arrays?

a) difficulty in knowing children nodes of a node

b) difficult in finding the parent of a node

c) have to know the maximum number of nodes possible before creation of trees

d) difficult to implement

3. What must be the ideal size of array if the height of tree is 'l'?

a) 2^l-1

b) l-1

c) l

d) 2l

Explanation: Maximum elements in a tree (complete binary tree in worst case) of height 'L' is 2^L-1. Hence size of array is taken as 2^L-1.

1. What is the maximum number of children that a binary tree node can have?

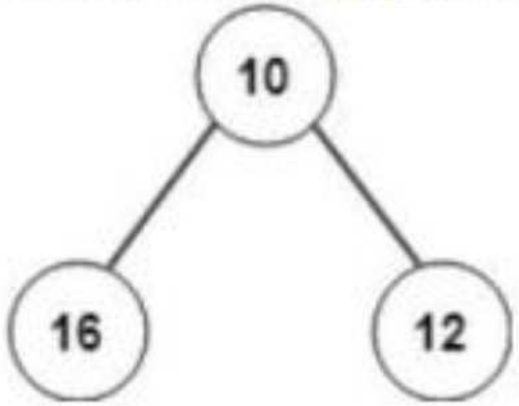
a) 0

b) 1

c) 2

d) 3

2. The following given tree is an example for?



a) Binary tree
b) Binary search tree
c) Fibonacci tree
d) AVL tree

4. What does the following piece of code do?

```
public void func(Tree root)
{
        func(root.left());
        func(root.right());
        System.out.println(root.data());
}
```

a) Preorder traversal
b) Inorder traversal
c) Postorder traversal
d) Level order traversal

TEACHING WALLAH

THANK YOU